

Contrôle :

La programmation orientée objet

Exercice 1 : *apprendre les getters/setters/le rôle d'héritage.*

1. Ecrire une classe Rectangle en langage Python, permettant de construire un rectangle doté d'attributs **longueur** et **largeur**.
2. Créer une méthode Perimetre() permettant de calculer le périmètre du rectangle et une méthode Surface() permettant de calculer la surface du rectangle
3. Créer les getters et setters.
4. Définir l'héritage.
5. Créer une **classe fille Parallelepipede héritant** de la classe Rectangle et dotée en plus d'un **attribut hauteur** et d'une autre **méthode Volume ()** permettant de calculer le volume du Parallélépipède.
6. Créer deux objets :
 - ✚ Un objet dans la 1^{er} classe puis à l'aide des deux fonctions prédéfinies calculer le périmètre et la surface.
 - ✚ Un objet dans la deuxième classe puis calculer le volume à l'aide de la fonction volume().

Exercice2 : *s'approfondir sur le principe de POO.*

1. Créer une classe Python nommée CompteBancaire qui représente un compte bancaire, ayant pour attributs : numeroCompte (type numérique) , nom (nom du propriétaire du compte du type chaîne), solde.
2. Créer un constructeur ayant comme paramètres : numeroCompte, nom, solde.
3. Créer une méthode Versement() qui gère les versements.
4. Créer une méthode Retrait() qui gère les retraits.
5. Créer une méthode Agios() permettant d'appliquer les agios à un pourcentage de 5 % du solde
6. Créer une méthode afficher() permettant d'afficher les détails sur le compte
7. Donner le code complet de la classe CompteBancaire.

Exercice 3 : *L'objectif de cet exercice c'est la distinction entre un constructeur et destructeur .*

- 1- Définir un destructeur.
- 2- Créer une classe Monobjet , puis la fonction constructeur (__init__) qui permet de créer un attribut **valeur** et affiche ("création de l'objet avec la valeur ",**self.valeur**)
- 3- Essayer de créer 3 objets puis déstructurer 2 premiers objets et essayer d'afficher la valeur du troisième objet et de l'un des objet pré-déstructuré.
- 4- On constatera que la valeur du troisième est affichée tandis que celle du deuxième N'est pas affichée.
Un message d'erreur est illisible par l'utilisateur est affiché
Cette fois-ci on veut utiliser la bonne-pratique pour indiquer justement le type d'erreur(<class 'NameError'>) toute en utilisant **try/except**.

Exercice 4 : le polymorphisme

1- Indiquer l'utilité de polymorphisme.

2-Créez une classe `Animal` qui possède un attribut `nom` et une méthode `parler()`. La méthode `parler()` devrait afficher "Je suis un animal et je ne parle pas." par défaut.

3-Créez une classe `Chien` qui hérite de `Animal` et qui possède une méthode `parler()` redéfinie. La méthode `parler()` de `Chien` devrait afficher "Ouaf!"

4-Créez une classe `Chat` qui hérite également de `Animal` et qui possède une méthode `parler()` redéfinie. La méthode `parler()` de `Chat` devrait afficher "Miaou!"

5-Créez des instances de `Chien` et de `Chat` et stockez-les dans des variables `mon_chien` et `mon_chat`.

6-Appellez la méthode `parler()` sur chacune de ces instances en utilisant un boucle `for`. Vous devriez voir que la méthode `parler()` de chaque instance est appelée et que le bon message est affiché, en utilisant le polymorphisme.

EXERCICE5 : la composition

En programmation orientée objet, la composition consiste à utiliser des objets existants dans un autre objet afin de créer de nouvelles fonctionnalités. Cela permet de réutiliser du code existant et de le combiner de manière flexible pour créer de nouvelles classes.

Partie1 :

1. Créez une classe `Voiture` qui possède un attribut `marque` et une méthode `demarrer()`. La méthode `demarrer()` devrait afficher "La voiture démarre."
2. Créez une classe `Personne` qui possède un attribut `nom` et une méthode `conduire()`. La méthode `conduire()` devrait accepter un objet `Voiture` en argument et appeler sa méthode `demarrer()`.
3. Créez une instance de `Voiture` et stockez-la dans une variable `ma_voiture`.
4. Créez une instance de `Personne` et stockez-la dans une variable `moi`.
5. Appelez la méthode `conduire()` de `moi` en lui passant `ma_voiture` en argument. Vous devriez voir que la méthode `demarrer()` de `ma_voiture` est appelée et que le message "La voiture démarre." est affiché.

Partie 2 :

1. Créez une classe `Oeuf` qui possède un attribut `couleur` et une méthode `frais()` qui retourne `True` si l'oeuf est frais, et `False` sinon. Par défaut, l'oeuf est frais lorsqu'il est créé.
2. Créez une classe `Poule` qui possède un attribut `nom` et un attribut `oeuf`, qui est un objet de type `Oeuf`. La classe `Poule` devrait également posséder une méthode `poser_oeuf()` qui crée un nouvel oeuf en utilisant la classe `Oeuf` et qui le stocke dans l'attribut `oeuf` de la poule.
3. Créez une instance de `Poule` et stockez-la dans une variable `ma_poule`.
4. Appelez la méthode `poser_oeuf()` de `ma_poule` et vérifiez que l'attribut `oeuf` de la poule contient un nouvel oeuf frais en appelant la méthode `frais()` de l'oeuf.

Exercice 6 : le surcharge

Partie 1 :

1. Créez une classe `Vecteur2D` qui représente un vecteur dans l'espace à deux dimensions. La classe devrait posséder deux attributs `x` et `y` qui représentent les coordonnées du vecteur.
2. Implémentez l'opérateur `+` pour que les instances de `Vecteur2D` puissent être additionnées entre elles. L'opérateur `+` devrait retourner un nouvel objet `Vecteur2D` qui est la somme des deux vecteurs.
3. Implémentez l'opérateur `*` pour que les instances de `Vecteur2D` puissent être multipliées par un scalaire. L'opérateur `*` devrait retourner un nouvel objet `Vecteur2D` qui est le produit du vecteur par le scalaire.
4. Créez deux instances de `Vecteur2D` et stockez-les dans des variables `v1` et `v2`.
5. Ajoutez `v1` et `v2` en utilisant l'opérateur `+` et stockez le résultat dans une variable `v3`.
6. Multipliez `v3` par un scalaire en utilisant l'opérateur `*` et stockez le résultat dans une variable `v4`.

Partie :2

1. Créez une classe `CompteBancaire` qui possède un attribut `solde` et une méthode `__init__()` qui initialise le solde à 0. La classe devrait également posséder une méthode `deposer()` qui permet de déposer de l'argent sur le compte et une méthode `retirer()` qui permet de retirer de l'argent du compte.
2. Implémentez l'opérateur `+` pour que les instances de `CompteBancaire` puissent être additionnées entre elles. L'opérateur `+` devrait déposer le montant de l'autre compte sur le compte courant.
3. Implémentez l'opérateur `-` pour que les instances de `CompteBancaire` puissent être soustraites entre elles. L'opérateur `-` devrait retirer le montant de l'autre compte du compte courant.
4. Créez deux instances de `CompteBancaire` et stockez-les dans des variables `compte1` et `compte2`.
5. Ajoutez `compte2` à `compte1` en utilisant l'opérateur `+`.
6. Soustrais `compte2` à `compte1` en utilisant l'opérateur `-`.